

Summary of Inference 2/12/2026

Barry Lyu(barrylyu),Jingjing Zhang(zjjing), Yile Sun(sunyyyl), Zhengqing Liu(zhengql)

DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving

Problem and Motivation

The core challenge in serving Large Language Models lies in the distinct computational characteristics of its two phases: the prefill phase, which processes the input prompt, and the decoding phase, which generates output tokens autoregressively. Existing serving systems typically colocate these two phases on the same GPUs and use continuous batching to maximize overall system throughput. However, this strategy leads to significant interference because the compute-intensive prefill tasks often block the memory-bound decoding tasks. This causes sudden, random delays, making it difficult to guarantee fast start times (TTFT) and continuous generation speeds (TPOT).

Furthermore, colocating these phases couples their resource allocation and parallelism strategies, which is inefficient because the two phases have conflicting requirements. The prefill phase typically benefits from intra-operator parallelism to reduce execution time for satisfying TTFT, whereas the decoding phase favors inter-operator parallelism or simple replication to maximize throughput under TPOT constraints. Because existing systems are forced to choose a single parallelism strategy for both, they often have to allocate extra compute resources to meet the stricter of the two requirements. Thus, this paper proposes DistServe to disaggregate the prefill and decoding phases onto separate GPUs, enabling independent resource allocation and parallelism optimization to maximize per-GPU goodput.

Related Works

Most state-of-the-art serving systems, such as vLLM and Orca, focus on maximizing overall throughput through techniques like continuous batching and PagedAttention. While effective for throughput, these systems rely on colocating prefill and decoding, which results in the interference issues described above. Other approaches, such as SARATHI and DeepSpeed-MII, attempt to mitigate this interference using chunked-prefill techniques (piggybacking decoding steps with smaller prefill chunks). However, DistServe argues that chunking introduces extra memory overhead and does not fully eliminate the interference or the resource coupling problem.

Additionally, while there are existing works on goodput-optimized scheduling for deep learning (e.g., Pollux, Clockwork), these works primarily target training workloads or traditional, smaller DNNs rather than the specific autoregressive generation characteristics of LLMs. AlpaServe addresses LLMs but focuses on non-autoregressive generation. DistServe also acknowledges concurrent works like Splitwise, which similarly explore disaggregation. However, DistServe distinguishes itself by focusing specifically on optimizing "goodput" (rate of requests meeting SLOs) and by introducing a placement algorithm that explicitly accounts for cluster bandwidth constraints (e.g., handling limited cross-node bandwidth vs. high-speed intra-node NVLINK).

Solution Overview

DistServe addresses the inefficiencies of colocated LLM serving by disaggregating the prefill and decoding phases into separate GPU instances. DistServe creates two independent instance types: prefill instances that process full prompt sequences to produce the first token, and decoding instances that generate subsequent tokens step-by-step using KV caches received from prefill. Prefill and decoding have fundamentally different performance bottlenecks. Prefill is typically compute-bound once prompt length exceeds a saturation threshold, meaning additional batching increases latency without improving throughput. Decoding, in contrast, is bandwidth-bound due to repeated KV cache reads and model weight accesses, making it highly sensitive to batch size and memory traffic. By separating the phases, DistServe can independently choose intra-operator parallelism (tensor parallelism), inter-operator parallelism (pipeline parallelism), and replication factors tailored to each phase's bottleneck.

DistServe's placement and configuration algorithm jointly integrates phase-specific performance characteristics, bandwidth constraints, and simulation-based evaluation into a constrained search procedure. The algorithm first decomposes the problem by optimizing prefill and decoding separately, since they have fundamentally different bottlenecks and SLO objectives. For each phase, it enumerates candidate configurations defined by intra-operator parallelism, inter-operator parallelism, batching, and per-instance GPU allocation, and evaluates them with respect to phase-specific latency constraints (TTFT for prefill, TPOT for decoding). Prefill configurations are assessed based on the tradeoff between reducing execution time and increasing service capacity, while decoding configurations prioritize large batching and memory-bandwidth utilization to minimize queuing delay. This phase-aware enumeration defines what configurations are worth considering, but does not yet account for physical deployment constraints.

The algorithm then incorporates bandwidth-aware placement constraints and simulation-based evaluation to select the optimal configuration. Because disaggregation introduces KV-cache transfers, cluster topology directly constrains feasibility: in high-bandwidth clusters, prefill and decoding instances may be placed independently, whereas in low node-affinity clusters, corresponding pipeline stages must be colocated to ensure KV transfers occur over high-bandwidth links such as NVLINK. Configurations violating these constraints are pruned before evaluation. For each remaining feasible configuration, DistServe runs a discrete-event simulator that models execution latency, memory access cost, and queuing behavior under

realistic workload traces to estimate TTFT and TPOT distributions. Using these distributions, the algorithm computes the maximum arrival rate that satisfies target SLO attainment and selects the configuration that maximizes per-GPU goodput, after which replication is applied to scale to the desired traffic rate.

Limitations

DistServe is explicitly designed for goodput-optimized, large-scale LLM serving. In throughput-optimized scenarios where latency is not a primary concern—such as offline or batch inference—maximizing raw throughput and GPU utilization may be more important than SLO attainment. In these cases, techniques such as chunked prefill with piggybacking can better saturate compute resources by filling batches to the compute-bound threshold, potentially outperforming DistServe’s disaggregated design. Similarly, in resource-constrained environments with only a few GPUs or a single device, DistServe’s design space is significantly limited. The system may be unable to meaningfully adjust parallelism or replication, and the overhead of disaggregation can outweigh its benefits, making simpler, non-disaggregated serving architectures more practical.

DistServe also has limitations in placement flexibility and evaluation scope. In low node-affinity clusters, the system mitigates KV-cache transfer overhead by colocating corresponding pipeline stages of prefill and decoding, partially reintroducing spatial coupling between phases and reducing the benefits of full disaggregation. Additionally, the ablation studies primarily focus on identifying and addressing the interference caused by prefill–decoding colocation, but do not systematically evaluate throughput tradeoffs across different optimization targets. Finally, while DistServe remains promising for long-context serving, where prefill cost grows quadratically and decoding interference becomes more severe, KV-cache transmission overhead still grows linearly with context length and may introduce new bottlenecks at extreme scales.

Future Research Directions

Future research could extend DistServe’s optimization framework to explicitly account for varying context window sizes, enabling adaptive batching and chunking strategies that better balance GPU utilization across short- and long-context requests. Incorporating fairness-aware scheduling to improve service quality under multi-tenant workloads. In addition, as agentic and multi-step inference workloads become more prevalent, future systems should explore more sophisticated scheduling policies and runtime optimization mechanisms.

Summary of Class Discussion

1. What is the difference between optimizing Goodput and Throughput? Why doesn't the paper report total throughput metrics?

Throughput refers to the total number of requests processed per unit of time, whereas Goodput specifically refers to the throughput that finishes within a specific time bound (Service Level Agreement/SLA).

If a request misses the SLA (e.g., takes too long), the provider might not get paid or gets paid less, so maximizing Goodput is maximizing revenue. The DistServe paper focuses on Goodput because they prioritize meeting strict latency constraints over raw processing volume, which might be lower than other methods but is more valuable for latency-sensitive applications.

2. Does disaggregating the Prefill and Decode phases (separating them onto different GPUs) cause significant communication overhead, especially if cross-node communication is slow?

While cross-node communication (transferring KV Cache) has overhead, the authors evaluated scenarios with both high and low bandwidth (affinity) and found the results were sustained even with slower communication.

The speakers noted that modern cloud instances (like AWS P5/B200) have massive bandwidth (e.g., 3.2 Terabytes per second), which is many times higher than what was evaluated. This high bandwidth largely mitigates the latency penalties of disaggregation. Strategy for Low Bandwidth: In environments with slower communication, DistServe doesn't just split across random machines. It may share the same GPU at a more granular level, such as dividing by Stages.

3. How do you determine the optimal number of GPUs to allocate? (To avoid allocating extra resources while maintaining performance).

Providers often use too many GPUs to handle the imbalance and interference between phases. By separating the phases, DistServe attempts to optimize them independently using fewer resources.

NVIDIA is addressing this by creating Rubin architecture GPU types (memory-heavy vs. compute-heavy) and orchestrators like NVIDIA Dynamo. This software automatically plans how many prefill vs. decode instances are needed based on load and auto-scales them dynamically.

4. Is synchronization harder when separating Prefill and Decode across different machines compared to using a single GPU? How are rate mismatches handled?

Yes, synchronizing across distributed machines is harder than on a single GPU due to potential rate mismatches (e.g., Prefill producing faster than Decode can consume).

Solutions:

1. Producer-Consumer Pattern: Systems use a software layer to buffer data between the stages.
2. Backpressure: If the receiver (Decode) is full, it pushes back or simply stops "pulling" data, signaling the sender (Prefill) to stop producing.
3. Load Balancing: A load balancer distributes requests to queues that have space; if all queues are full, it triggers auto-scaling to add more capacity.

Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve

Problem and Motivation

Serving online LLM inference service with high cost efficiency is difficult as there is a fundamental conflict between service level constraints for user experience, including time-between-tokens (TBT) and time-to-first-token (TTFT), which are latency oriented, and GPU utilization, which is throughput oriented. This throughput-latency tradeoff is centered around the imbalance between decode and prefill: prefill is compute-heavy and can saturate GPU utilization well, while decode is memory bandwidth-bound, leaving GPUs underutilized. Batching decode can increase memory reuse, but will increase the latency of individual requests: Larger decode batches can achieve high arithmetic intensity, enhancing throughput significantly, but this results in longer waits to fill the batch. Smaller decode batches will manifest lower latency but not fully utilize the hardware, decreasing the overall throughput. Hence, careful selection of batching opportunities is crucial to meeting SLO with high efficiency.

In addition, continuous batching can result in poor latency and "generational stalls", where long prefills can block or delay the decode of an active request, resulting in unstable token streaming and hurting the interactive user experience. These iteration variations may also disrupt the pipeline schedule, creating bubbles that also reduce the GPU utilization and hence the overall efficiency. In pipeline-parallel inference, micro-batch runtimes vary widely depending on how many prefill vs decode tokens they contain (and cache length), causing stages to idle and creating pipeline bubbles that waste GPUs.

Related Works

Prior works either prioritize decodes to maintain high token streaming stability, giving up opportunities for aggressive batching, or prioritize prefill over decode, sacrificing user experience for higher service efficiency. Decode-prioritizing designs such as FastTranformer keep TBT stable but suffer shrinking batches and lower throughput, while iteration-level continuous batching systems like Orca or vLLM improve throughput but can exhibit stalls when prefills interrupt decode.

Some systems support hybrid batches with mixed prefill and decodes, which does not prevent stalls due to long prompts dominating the iteration runtime: If hybrid batches fuse an entire long prefill with decodes, the iteration length becomes dominated by the prefill, delaying the next decode step and recreating the stall problem.

DistServe and its line of work disaggregates prefill and decode onto replicated instances to reduce the prefill-decode interference, which can be very effective with strict latency targets. However, P/D disaggregation requires KV-Cache transfers, which relies on fast transfer paths (or clever placement) for KV-cache movement and can constrain how GPU memory is utilized across the prefill and decode replicas.

Solution Overview

Sarathi-Serve aims to preserve the throughput benefits of continuous batching while preventing decode stalls and reducing pipeline inefficiency. They observe that carefully combining prefill and decode may improve overall throughput without latency degradation, as decode iteration will not fully saturate compute throughput. Mixing prefill workloads with controlled latency may help to fill the remaining capability of the GPU, along with pipeline bubbles, with minimal impact on decode latency.

Sarathi-Serve combines two ideas: (1) chunked prefills and (2) stall-free batching, to make hybrid batches predictable and decodable. Sarathi opens up more opportunities for scheduling prefill by introducing chunked prefills and breaks up the entire prompted prefill into smaller chunks. This allows for fine-grained scheduling of prefill jobs, but introduces another tradeoff: Larger prefill chunks would reintroduce stalls, while small prefill chunks would incur too much scheduling overhead and increase HBM traffic compared to a monolithic prefill, as prior context needs to be read repeatedly from KV-Cache. Sarathi also notes the tile quantization effects on GPUs, where misaligned tiling can lead to disproportionate slowdowns, and select chunk sizes with awareness of these performance degradations.

On top of chunking, Sarathi-Serve uses a stall-free scheduling policy driven by that token budget: It first identifies the maximum token budget τ for a given TBT SLO via profiling, then chooses the largest chunk that can fit inside the iteration. The scheduler always admits all decode tokens for active requests first, then adds partially completed prefill chunks, and only admits new requests if there is remaining budget. This admission policy thus guarantees that new prefills cannot preempt or delay decoding beyond the controlled budget. A further benefit is that by bounding and stabilizing per-iteration work, Sarathi-Serve reduces runtime variance across micro-batches, which helps mitigate pipeline bubbles in pipeline-parallel deployments and improves multi-GPU efficiency.

Limitations

By breaking a large prefill request into smaller chunks and interleaving them with ongoing decoding tasks, the system effectively spreads the prompt processing time over multiple iterations. While this improves inter-token latency, it inevitably increases the TTFT for the new request compared to a system that monopolizes the GPU to process the prompt in a single burst.

Despite mitigating interference via software scheduling, the system retains the limitation of a colocated architecture, where the prefill and decode phases compete for the same physical resources. Unlike disaggregated systems (e.g., DistServe), Sarathi-Serve cannot scale the compute resources for prefill independently of the memory resources for decoding.

An evaluation flaw in the paper is the reliance on overly relaxed service level objectives (SLOs) to demonstrate success. The authors define a "relaxed" latency objective with a P99 Time-Between-Tokens (TBT) of 5 seconds, which is impractically high for real-world interactive applications. Furthermore, the evaluation focuses on optimizing tail latency (P99 TBT) while neglecting the impact on median latency.

Future Research Directions

A possible direction is to combine Sarathi-Serve's chunk prefills with the architectural disaggregation of DistServe. While DistServe physically separates prefill and decode phases to isolate interference, the prefill cluster can still suffer from bursty output, overwhelming the network bandwidth during the KV cache transfer to decode nodes. Applying Sarathi-Serve's chunking mechanism within the prefill workers of a disaggregated system could smooth out this data flow.

Summary of Class Discussion

Q1: If two requests come at the same time, one is long and another is short, how does Sarathi-Serve balance these two prompts?

A1: Sarathi-Serve effectively addresses this through the token budget. By treating the GPU's capacity as a fixed budget of tokens per iteration, the system automatically fits as many prefill tokens as possible into the remaining space after scheduling decoding tasks. This chunking strategy ensures that long requests are processed incrementally, preventing them from monopolizing the GPU.

This question introduces the problem of fair sharing across multiple requests. In a strict FCFS queue, a single long request can monopolize the hardware, forcing all subsequent requests to wait regardless of their size. While this approach is technically "fair" regarding arrival time, it creates a poor user experience by stalling short tasks behind long ones.

To resolve this, some other works propose a higher-level scheduler that operates not at the granularity of whole requests, but at an intermediate level. By implementing a Round-Robin or fair-sharing policy at this finer granularity, the system ensures that long requests yield resources periodically, preventing them from blocking the queue indefinitely.

A mechanism discussed for enforcing this fairness is the use of time quanta. Instead of running a request to completion, the scheduler assigns a specific quantum of work (a fixed time slice or

token limit) to each active user. Large requests consume their quantum and are then cycled to the back of the priority queue.

Finally, the professor mentions that modern LLM serving systems are not single queues but complex networks of queues. Scheduling decisions happen simultaneously at multiple layers. Because data flows dynamically between these layers, theoretical analysis of the system's behavior is difficult. They rely on empirical experimentation to show that the interaction between layers produces the desired fairness and throughput and explain why.